

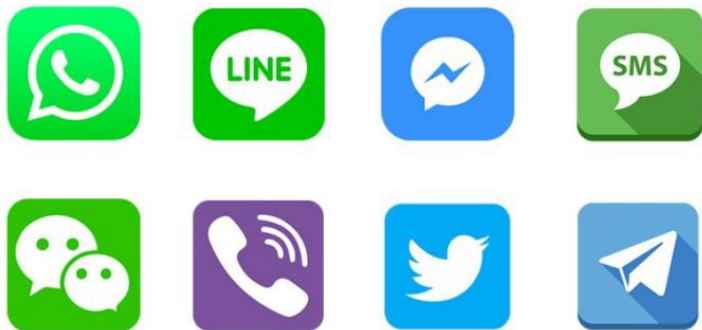
University of Stuttgart
Germany

Core design challenges in distributed systems

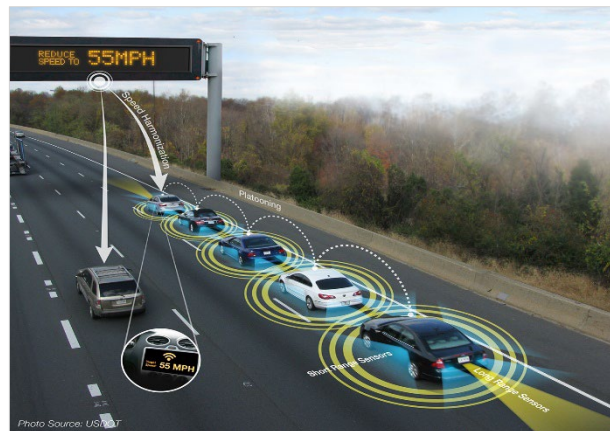
Real-world examples, case study,
key design challenges

Ilche Georgievski

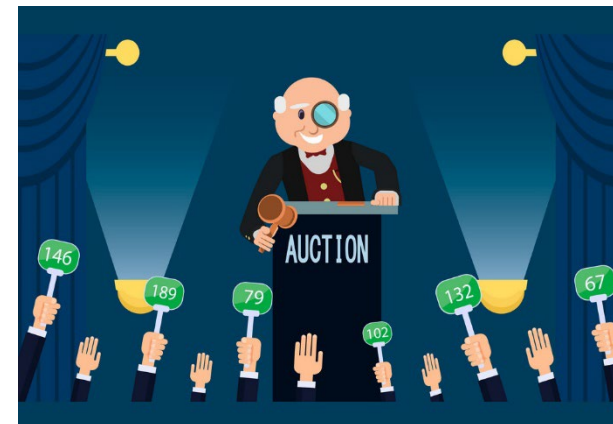
ilche.georgievski@iaas.uni-stuttgart.de



Chat applications



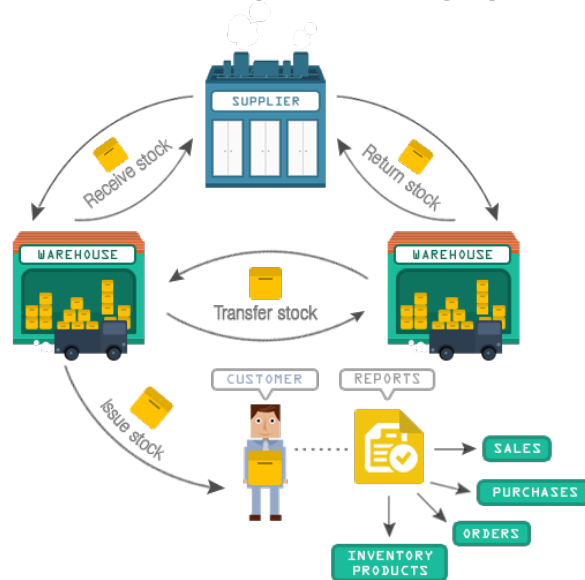
Vehicle platooning system



Online auction system



Live betting system



Inventory management system



Massive multiplayer online games

Chat application (I)

- An application that enables real-time transmission of messages over Internet

- Examples of challenges

Example of implementation choices

Architecture model

Client-server architecture, where chat participants are clients and system components are servers.

Dynamic discovery of hosts

New chat participant discovers the server providing a particular chat and joins the chat. A new system component discovers existing components and joins the system.

Leader election (voting)

LCR algorithm is used to designate a system component responsible for the authoritative and coordination tasks.

- The system may be composed of either distinctive components such as clients and servers (e.g., WhatsApp) or equipotent components, that is, peers (e.g., Tox)
- How can new chat participants join dynamically without restarting or reconfiguring?
- How can new system components join dynamically (e.g., scaling out chat rooms) while preserving availability?
- New chat participants may need to be accepted/approved by an authoritative component
- System components may need to be checked whether they are still up and running by some coordinator

Chat application (2)

- An application that enables real-time transmission of messages over Internet

- Examples of challenges

- Should message reliability be guaranteed? What happens if a message gets lost? How important it is?
- Should the order of messages be the same at all chat participants? How important it is to display the messages in the proper order?
- Different mechanisms for exchanging messages may be needed for different aspects of the system (e.g., message ordering needs may be different at a chat level and at a system level)

- What happens if a chat participant becomes unavailable? How important it is?
- What happens if a system component becomes unavailable? How important it is?

Example of implementation choices

Reliable ordered multicast

If each chat participant must receive every message, reliable multicast is needed.

For messages to be received in the same order as sent by chat participants, FIFO ordering is needed at least. To have a chat that preserves the relationship between messages, **causal ordering** is needed.

If system components store replicated chat data, reliable multicast is needed at least.

Fault tolerance

Detecting and handling crash failures of a leader and servers.

Online auction system

- Scenario
 - The system enables users to create auctions, register as bidders, and place bids in real time.
 - The system runs on multiple interconnected components that may fail, recover, or join dynamically.
 - Key goals: consistency, reliability, fault tolerance, and correct functioning of the system as a whole.
- Examples of challenges
 - Should the system be composed of specialised components (e.g., auctioneer, bidder) or equipotent peer nodes? What are the trade-offs for coordination, scalability, and fault tolerance?
 - How should bidders join or leave an auction without disrupting ongoing bidding?
 - How can new system components (e.g., new auction servers) be added or removed while maintaining system consistency? What mechanisms are needed to detect and handle membership changes?
 - Who has the authority to change an auction's status (e.g., from open to closed)?
 - How should incoming bids be ordered consistently across replicas? How do all nodes agree on the current highest bid?
 - How should the system elect or designate a leader responsible for managing a particular auction?
 - Will auction state be replicated across multiple nodes?
 - Should operations (e.g., place bid, close auction) be replicated and how to ensure all replicas apply operations in the same order?
 - How do we ensure that after a crash, the system still knows the most recent highest bid?
 - Should every participant receive all bids?
 - What kind of message ordering should be guaranteed among participants?
 - How do we ensure reliable multicast and avoid lost or duplicated bids under failures or network partitions?

Massive multiplayer online game

- An online video game that can be played by a very large number of people at the same time (e.g., Runescape)
- Examples of challenges
 - The system may be composed of functionally equipotent components
 - Will there be clusters of players depending on a player's location (world region)?
 - Where should be the game world stored?
 - How can a new player find and join a game (or a cluster of players)?
 - How can new system components be added to the system without interrupting gameplay?
 - How to ensure a consistent view of the game world?
 - How to ensure a consistent view of the system (global state) across many servers?
 - Is the order of actions performed by the players important? What guarantees are needed?
 - What happens when a server fails?

Designing a DS means reasoning about

- **Who participates?**
 - What roles and responsibilities system components have?
- **How they communicate?**
 - What communication guarantees are provided?
- **How they find each other?**
 - How participants discover, join, and leave the system?
- **What happens when they fail?**
 - How the system detects, recovers, and reconfigures after faults?
- **How consistent the system state must be?**
 - What guarantees user experience when concurrent updates occur?

These five aspects collectively define the system's **architecture, communication behaviour, membership dynamics, fault tolerance, and consistency.**

A DS involves multiple layers of complexity

- One or more distinctive roles or groups of participants
 - Example: servers, clients, coordinators
 - Example: server group 1, server group 2, client group 1, client group 2
- One or more mechanisms for exchanging messages
 - Example: basic multicast (server-to-server),
 - Example: reliable ordered multicast (server-to-client),
 - Example: broadcast (for host discovery)
- One or more points for discovery of hosts
 - Example: clients discover servers, servers discover peers, replicas discover coordinators
- One or more types of failures, at any role
 - Example: server crashed, leader crashes
 - Example: omission failures
 - Example: Byzantine failures

Thinking exercise

- Scenario
 - Imagine you are designing a **collaboration system** that allows millions of users to work together on shared 3D designs.
 - Users should be able to join or leave collaborative sessions at any time, see updates from others in real time, work offline temporarily and sync when they reconnect, have their data replicated across regions for low latency and fault tolerance.
- Reason about
 - Who participates?
 - How they communication?
 - How they find each other?
 - What happens when they fail?
 - How consistent the state must be?

This exercise helps you reason about design trade-offs using the same principles we have just discussed.